



APPLIED MATH FOR COMPUTER SCIENCE

CPCS 212

LAB 2 : Introduction to MATLAB

Agenda

- **Some Commonly Used Mathematical Functions**
- **Vectors**
- **Getting information about matrix:**
- **Data Types**
- **Managing the work session**
- **Script files**

Lab2

Open the diary file
Lab2_yourname.txt

Some Commonly Used Mathematical Functions

Function in calculus	MATLAB Syntax
e^x	exp(x)
$\sqrt{x}$	sqrt(x)
$\ln x$	log(x)
$\log_{10} x$	log10(x)
$ x $	abs(x)
$\cos x$	cos(x)
$\sin x$	sin(x)
$\tan x$	tan(x)
$\cos^{-1} x$	acos(x)
$\sin^{-1} x$	asin(x)
$\tan^{-1} x$	atan(x)

Note: The MATLAB trigonometric functions use radian measure

To convert from Degree $\rightarrow$ Radian

use :

$$180^\circ \rightarrow \pi$$

Ex:

$$60^\circ = 60 * \pi / 180 = \pi/3$$

Or:

use sind()

Common mathematical functions

Function	Description	Example	Result
ceil(x)	Round to nearest integer toward ∞ .	>>ceil(2.2)	ans = 3
fix(x)	Round to nearest integer toward zero.	>>fix(2.9) >>fix(-2.9)	ans = 2 ans = -2
floor(x)	Round to nearest integer toward $-\infty$.	>> floor(2.3)	ans = 2
round(x)	Round toward nearest integer.	>>round(2.3) >>round(2.5)	ans= 2 ans= 3
sign(x)	+1 if $x > 0$; 0 if $x=0$; -1 if $x<0$	>>sign (3) >>sign(-5) >> sign(0)	ans= 1 ans= -1 ans= 0

2-(a)

Choose an angle θ and set the variable **theta** to its value (a number).

```
>> % 2 - (a)
```

```
>> theta = 180
```

2-(b)

What is $\sin \theta$? $\cos \theta$? Angles can be measured in degrees or radians. Which of these has Matlab used?

```
>> % 2 – (b)
```

```
>> sin( theta)
```

```
>> cos(theta)
```

Note

- To work in radians, use `sin`, `cos`, `asin`, `acos` etc.
- To work in degrees, use `sind`, `cosd`, `asind`, `acosd` etc.
- To convert from degrees to radians, multiply by $\pi/180$.
- To convert from radians to degrees, multiply by $180/\pi$.

Answer:

```
>> sin( theta* pi/180)
```

```
>> cos(theta * pi/180)
```

or

```
>> sind( theta)
```

```
>> cosd(theta)
```

2-(c)

Matlab variables can also be given “character” or “string” values. A string is a sequence of letters, numbers, spaces, etc., surrounded by single quotes (').

In your own words, what is the difference between the following two expressions?

a1='sqrt(4) ' and a2=sqrt(4)

>> % 2 – (c)

>> a1 = 'sqrt(4) '

>> a2 = sqrt(4)

2-(d) - question

The Matlab *eval* function is used to EVALuate a string as if it were typed at the command line. If *a1* is the string given above, what is the result of the command *eval(a1)*?

Of $a3=5*eval(a1)$?

```
>> % 2 - (d)
```

```
>> a3 = 5 * eval(a1)
```

2-(e)

Use the *save* command to save all your variables. Check your “Current Directory” to see that you have created the file “lab2.mat”.

```
>> % 1 – (i)
```

```
>> save lab2
```

2-(f)

Use the *clear* command. Check that there are no variables left in the "current workspace".

```
>> % 2 – (f)
```

```
>> clear all
```

2-(g)

Restore all the variables with ***load*** and check that the variables have been restored to the “Current workspace”.

```
>> % 2 – (g)
```

```
>> load lab2
```

ASSIGNMENT 2- 1

- Evaluate the following expressions in Matlab:

1. $\pi - e^\pi$

2. $\log_{10}(2)$

3. $1/0$

4. $0/0$

5. $1 - 10^{-8}$

6. $1 - 10^{-20}$

Vectors :

- *A vector is a one-dimensional array of numbers. MATLAB allows you to create column vectors or row vectors using: Square bracket “[]”, Colon operator “:”, **linspace** and **logspace***
 - **Rowvector = [E1, E2, ..., Em] ,**
 - **Rowvector = [E1 E2 ... Em]**
 - **Columnvector= [E1;E2; ... ;Em]**
 - **Rowvector = first : step : last**
 - **Rowvector = first : last**
 - **linspace(x1,x2,n)**
 - **logspace(x1,x2,n)**

Linspace / logspace

`y = linspace(x1,x2)` returns a row vector of 100 evenly spaced points between `x1` and `x2`.

`y = linspace(x1,x2,n)` generates `n` points. The spacing between the points is $(x2-x1)/(n-1)$.

`y = logspace(a,b,n)` generates `n` points between decades 10^a and 10^b

`y = logspace(a,b)` generates a row vector `y` of 50 logarithmically spaced points between decades 10^a and 10^b .

Examples:

Input	Output		Comments
<pre>a=[1 2 3 4 5] b=[1:5] , c=1:5 d= linspace(1,5,5)</pre>	<pre>a=1 2 3 4 5 b=1 2 3 4 5 c=1 2 3 4 5 d=1 2 3 4 5</pre>		Creating same vector with different method
<pre>d = 0:2:10</pre>	<pre>d=0 2 4 6 8 10</pre>		
<pre>a(3) a(2:4) a(:)</pre>	<pre>ans = 3 ans = 2 3 4</pre>	<pre>ans=1 2 3 4 5</pre>	More uses of the colon
<pre>e = [1 2 3; 4 5 6; 7 8 9]</pre>	<pre>e= 1 2 3 4 5 6 7 8 9</pre>		Entering a matrix.
<pre>e(2,3) e(3,:) e(2,3) = 4</pre>	<pre>ans = 6 ans = 7 8 9 e = 1 2 3 4 5 4 7 8 9</pre>		Accessing matrix elements. Note use of ":" to access an entire row.

Examples:

Input	Output	Comments																		
<code>f = [a b]</code>	<code>f=1 2 3 4 5 1 2 3 4 5</code>																			
<code>g=[a;b]</code>	<code>g= 1 2 3 4 5</code> <code>1 2 3 4 5</code>																			
<code>>>X= linspace(0, 2*pi,6)';</code> <code>>>Y=sin(X);</code> <code>>>Z=cos(X);</code> <code>>>[XY Z]</code>	<code>ans =</code> <table> <tr> <td>0</td><td>0</td><td>1.0000</td></tr> <tr> <td>1.2566</td><td>0.9511</td><td>0.3090</td></tr> <tr> <td>2.5133</td><td>0.5878</td><td>-0.8090</td></tr> <tr> <td>3.7699</td><td>-0.5878</td><td>-0.8090</td></tr> <tr> <td>5.0265</td><td>-0.9511</td><td>0.3090</td></tr> <tr> <td>6.2832</td><td>0</td><td>1.0000</td></tr> </table>	0	0	1.0000	1.2566	0.9511	0.3090	2.5133	0.5878	-0.8090	3.7699	-0.5878	-0.8090	5.0265	-0.9511	0.3090	6.2832	0	1.0000	
0	0	1.0000																		
1.2566	0.9511	0.3090																		
2.5133	0.5878	-0.8090																		
3.7699	-0.5878	-0.8090																		
5.0265	-0.9511	0.3090																		
6.2832	0	1.0000																		

Lab2 -Exercise - part 2

Exercise 2:

>> % Exercise 2:

Define the following vectors and matrices:

```
rowVec1 = [ -1 -4 -9]
```

```
colVec1=[3  
          -6  
          9]
```

```
mat1 = [ 1 3 5  
        7 9 2  
        4 6 8 ]
```

2-(h)

You can multiply vectors by constants.

Compute

$$\text{colVec2} = (\pi/4) * \text{colVec1}$$

```
>> % 2-(h)
```

```
>> colVec2 = (pi/4) * colVec1
```

2-(i)

The cosine function can be applied to a vector to yield a vector of cosines.

Compute

$$\text{colVec2} = \cos(\text{colVec2})$$

Are these value you expect?

```
>> % 2 - (i)
```

```
>> colVec2 = cos( colVec2 )
```

2-(i)

You can add vectors.

Compute

$$\text{colVec3} = \text{colVec1} + \text{colVec2}$$

```
>> % 2 - (i)
```

```
>> colVec3 = colVec1 + colVec2
```


2-(k)

Try to Compute

$$\textit{result} = \textit{colVec1} + \textit{rowVec1};$$

```
>> % 2 - (k)
```

```
>>result = colVec1 + rowVec1;
```

Old version of Matlab was not allow you to do this operations (illegal). But the new versions allows you to add row vector with a column vector it results a matrix.

2-(I)

You can do row-column matrix multiplication.

Compute

$$\textit{colvec4} = \textit{mat1} * \textit{colVec1}$$

```
>> % 2 - (I)
```

```
>> colvec4 = mat1 * colVec1
```

2-(m)

A single quote following a matrix or vector indicates a transpose.

mat1Transpose = mat1'

rowVec2 = colVec3'

```
>> % 2 - (m)
```

```
>> mat1Transpose = mat1'
```

```
>> rowVec2 = colVec3'
```

Warning: The single quote really means the complex-conjugate transpose (or Hermitian adjoint). If you want a true transpose applied to a complex matrix you must use " . ' ".

Example :

```
>> x = rand(2,3)*i
```

```
x =
```

0 + 0.6324i	0 + 0.2785i	0 + 0.9575i
0 + 0.0975i	0 + 0.5469i	0 + 0.9649i

```
>> x'
```

```
ans =
```

0 - 0.6324i	0 - 0.0975i
0 - 0.2785i	0 - 0.5469i
0 - 0.9575i	0 - 0.9649i

```
>> x.'
```

```
ans =
```

0 + 0.6324i	0 + 0.0975i
0 + 0.2785i	0 + 0.5469i
0 + 0.9575i	0 + 0.9649i

Transposes allow the usual operations. You might find $\mathbf{u}^T \mathbf{v}$ a useful expression to compute the dot (inner) product $\mathbf{u} \odot \mathbf{v}$ (although there is a *dot* function in Matlab).

2-(n)

```
mat2 = mat1 * mat1' % symmetric matrix
```

```
rowVec3 = rowVec1 * mat1
```

```
dotProduct = colVec3' * colVec1
```

```
euclideanNorm = sqrt(colVec2' * colVec2)
```

```
>> % 2 - (n)
```

```
>>mat2 = mat1 * mat1'
```

```
>>rowVec3 = rowVec1 * mat1
```

```
>>dotProduct = colVec3' * colVec1
```

```
>>euclideanNorm = sqrt(colVec2' * colVec2)
```

2-(o)

Matrix operations such as determinant and trace are available, too.

```
determinant = det( mat1 )
```

```
tr = trace( mat1 )
```

```
>> % 2 - (o)
```

```
>> determinant = det( mat1 )
```

```
>> tr = trace( mat1 )
```

Determinant: In linear algebra, the **determinant** is a scalar value that can be computed from the elements of a square matrix,

Trace: is defined to be the sum of elements on the main diagonal.

2-(p)

You can pick certain elements out of a vector, too. Use the following command to find the smallest element in a vector *rowVec1*.

`min(rowVec1)`

`>> % 2 - (p)`

`>> min(rowVec1)`

2-(q)

The *min* and *max* functions work along one dimension at a time. They produce vectors when applied to matrices.

max(mat1)

```
>> % 2 - (q)
```

```
>> max(mat1)
```


2-(r)

How would you find the single largest element of a matrix?

```
>> % 2 - (r)
```

```
>> max(max(mat1))
```

2-(s)

Consider the matrix

$$A = \text{magic}(5);$$

The matrix A has 5 row sums (one for each row), 5 column sums (one for each column) and two diagonal sums. These 12 sums should all be exactly the same, Verify

```
>> % 2 - (s)
```

```
>> A=magic(5);
```

2-(t)

- Suppose we want a table of integers from 0 to 10, their squares and cubes.

1. `integers = 0 : 10`

2. `SquareIntegers = integers * integers`

Wrong XXX

3. `squareIntegers = integers .* integers`

Using (.) to multiply each element in the vector with the corresponding one in the other vector. The result is vector

4. `cubeIntegers = squareIntegers .* integers`

5. `fourthIntegers = squareIntegers .* squareIntegers`

6. `tableOfPowers=[integers', squareIntegers',
cubeIntegers', fourthIntegers']`

Answer:

```
>> % 2 - (t)
>> integers = 0 : 10;
>> squareIntegers = integers .* integers;
>> cubeIntegers = squareIntegers .* integers;
>> fourthIntegers =
    squareIntegers.*squareIntegers;

>> tableOfPowers=[integers', squareIntegers',
    cubeIntegers',
    fourthIntegers']
```

2-(t)- homework

- Put the answer of question 2(t) in script file with name **q2m.m**
- And run the script file

2-(u)

Compute the squares of the values in integers alternatively using the exponentiation operator as:

sqIntegers = *integers* .^ 2

check that the two calculations agree with the command:

norm(sqIntegers-squareIntegers)

which should result in zero.

>> % 2 - (u)

Using (.) to find the square for each element in the vector. The result is vector

>>sqIntegers = integers .^ 2

>>norm(sqIntegers-squareIntegers)

2-(v)

```
tableOfCubes = tableOfPowers(:,[1,3])  
tableOfOddCubes = tableOfPowers(2:2:end,[1,3])  
tableOfEvenFourths = tableOfPowers(1:2:end,1:3:4)
```

```
>> % 2 - (v)
```

```
>> tableOfCubes = tableOfPowers(:, [1, 3])  
>> tableOfOddCubes =  
           tableOfPowers(2:2:end, [1, 3])  
>> tableOfEvenFourths =  
           tableOfPowers(1:2:end, 1:3:4)
```

2-(w) - homework

You have already seen the Matlab function *magic(n)*. Use it to construct a 10 x 10 matrix.

$$A = \text{magic}(10)$$

What commands would be needed to generate the four 5x 5 matrices in the upper left quarter, the upper right quarter, the lower left quarter, and the lower right quarter of A?

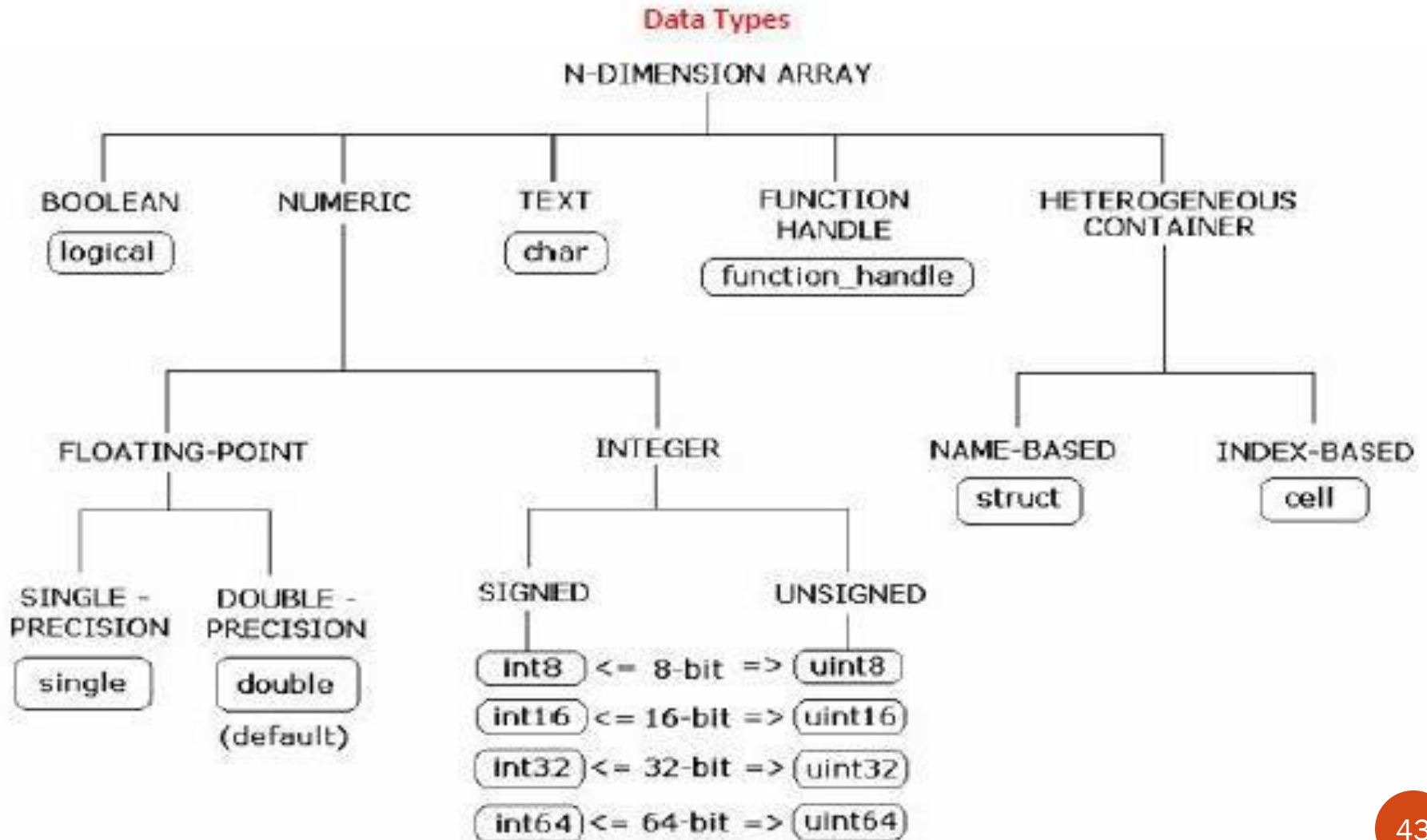
Getting information about matrix:

Command	Description	Example	Output
<code>[m n] = size(A)</code>	Return the size of matrix A where m= #row , n= # column	<pre>>> A= [1 2 3; 4 5 6]; >> [m n] = size(A);</pre>	<pre>m = 2 n = 3</pre>
<code>length(A)</code>	Returns the length of vector x or largest array A dimension equivalent to <code>MAX(SIZE(A))</code>	<pre>>> x=[1 5 8 4]; >> length(x) >> length(A)</pre>	<pre>ans= 4 ans= 3</pre>
<code>numel(A)</code>	Returns the number of elements in array A	<pre>>> numel(A)</pre>	<pre>ans= 6</pre>
<code>ndims(A)</code>	Returns the number of dimensions in the array	<pre>>> ndims(A)</pre>	<pre>ans= 2</pre>
<code>isempty(A)</code>	Determine whether A is empty	<pre>>> isempty(A)</pre>	<pre>ans= 0</pre>

Getting information about matrix:

Command	Description	Example	Output
isscalar(A)	Determine whether A is scalar	>>isscalar(A)	ans= 0
isvector(A)	Determine whether A is a vector	>>isvector(A)	ans= 0
isrow(A)	Determine whether A is row vector	>> isrow(A)	ans= 0
iscolumn(A)	Determine whether A is column vector	>> iscolumn(A)	ans= 0
ismatrix(A)	Determine whether A is matrix	>> ismatrix(A)	ans= 1
isequal(A,B)	Test A and B for equality	>> isequal(A,A) >> isequal(A,2*A)	ans= 1 ans= 0

Data Types



Data Types

Command	Description	Example	Output			
single(x)	Convert to single precision	>> single(5)	ans = 5			
		>> whos	Name	Size	Bytes	Class
			ans	1 x 1	4	single
double(x)	Convert to double precision	>> double(5)	ans=5			
		>> whos	Name	Size	Bytes	Class
			ans	1 x 1	8	double
int8(x)	Convert to signed integer	>> a= int8(x)	Name	Size	Bytes	Class
int16(x)		>> b= int16(x)	a	1 x 1	1	int8
int32(x)		>> c= int32(x)	b	1 x 1	2	int16
int64(x)		>> d= int64(x)	c	1 x 1	4	int32
		>> whos	d	1 x 1	8	int64
uint8(x)	Convert to unsigned integer	>> a= uint8(x)	Name	Size	Bytes	Class
uint16(x)		>> b=uint16(x)	a	1 x 1	1	uint8
uint32(x)		>> c=uint32(x)	b	1 x 1	2	uint16
uint64(x)		>> d=uint64(x)	c	1 x 1	4	uint32
		>> whos	d	1 x 1	8	uint64

Data Types

Command	Description	Example	Output			
char(x)	Convert to character using ASCII code	>> char(60)	ans = <			
		>> whos	Name	Size	Bytes	Class
			ans	1 x 1	2	char
@function name	Anonymous Functions Constructs handle (pointer) to function name	>> a=[1 2 ; 3 4]; >> h=@numel; >> h(a)	ans = 4			
@(input_arguments) function_code	Constructs handle (pointer) to function name	>> h=@(a,b) a+b; >> h(1,2) >> sqr = @(x) x.^2; >> a = sqr(5)	ans = 3 a = 25			

Getting information about data type

Command	Description	Example	Output
islogical(x)	Determine whether x is logical	>> a=5; >> islogical(a) >> islogical(a>2)	ans = 0 ans = 1
isnumeric(x)	Determine whether x is numeric	>> a=5; >> isnumeric(a) >> isnumeric(char(a))	ans= 1 ans=0
isinteger(x)	Determine whether x is integer	>> isinteger(a) >> isinteger(int8(a))	ans=0 ans=1
isfloat(x)	Determine whether x is floating point	>> isfloat(a) >> isfloat(int8(a))	ans=1 ans=0
isreal(x)	Determine whether x is real number	>> isreal(a) >> isreal(a*j)	ans=1 ans=0

Managing the work session

Command	Description
<code>clc</code>	Clears the command window
<code>clear</code>	Removes all variables from memory
<code>clear var1, var2</code>	Removes variables var1, var2 from memory
<code>save filename</code>	Save all workspace to filename
<code>Save filename var1 var2 var3</code>	Save workspace variables var1, var2, var3 to filename

Managing the work session

Command	Description
exist <i>name</i>	Determine if the file or variable exists having the name <i>name</i>
who	Lists the variables currently in memory
whos	Lists the current variables and sizes and indicates if they have imaginary part
...	Ellipsis continues a line
quit	Stop MatLab

Managing the work session

Expression or command	Example	Result												
Expression press enter	>> a=2	a = 2												
Expression ;	>> b=3;													
who	>> who	Your variables are: a b												
whos	>> whos	<table><tr><td>Name</td><td>size</td><td>Bytes</td><td>class</td></tr><tr><td>a</td><td>1×1</td><td>8</td><td>double</td></tr><tr><td>b</td><td>1×1</td><td>8</td><td>double</td></tr></table>	Name	size	Bytes	class	a	1×1	8	double	b	1×1	8	double
Name	size	Bytes	class											
a	1×1	8	double											
b	1×1	8	double											
clear var	>> clear a >> who	Your variables are: b												
exist var or file	>> exist a >> exist b	ans = 0 ans = 1												

49

Managing the work session

Expression or command	Example	Result
clear all	>>clear all >> who	
save file a b	>>a=1;b=2;c=3; >>save myfile a b c >>dir	myfile.mat
save file	>>save myfile	
load file	>>clear all >>load myfile >>who	Your variables are: a b c
clc	>> clc	>>
quit	>> quit	stop matlab

Lab 2

Lab2-Exercise-part 3

2-(x)

>> % **Exercise 3:**

Use the *linspace* function to create a row vector called *meshPoints* containing exactly 200 values with values *evenly spaced* between *-1* and *1*. Do not print all 200 values!

>> % **2 - (x)**

>> **meshPoints = linspace(-1, 1, 200);**

2-(y)

What expression will yield the value of the 53th element of *meshPoints*? What is this value?

```
>> % 2 - (y)
```

```
>> meshPoints (53)
```

Double-click on the variable *meshPoints* in the

"Current workspace" to view it as a vector and confirm its length is 200.

2-(z)

Use the ***length*** function to again confirm the vector has length 200.

```
>> % 2 - (z)
```

```
>> length(meshPoints)
```

Exc

- Produce a plot of a sinusoid on the interval $[-1, 1]$ using the command:

```
>> %
```

```
>> plot(meshPoints, sin(2*pi*meshPoints))
```

- Please save (export) this plot as a jpeg (.jpg) file and send it along with your summary. The command line expression to save a plot is:

```
>> print -djpeg plotname.jpg
```

Lab2-Exercise 3 -Script

- Create a file named *lab2exer3.m*. Type the previous commands into the window and use “Save as” to give it a name, or you can highlight some commands in the “command history” window and use a right mouse click to save the commands as an m-file. The first lines of the file should be the following:

% Lab 2, exercise 3

% A sample script file.

% Your name and the date

- Test your script by using *clear* to clear your results and then execute the script .

>> % 3 – (f)

>> clear